

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 1 REPORT

CRN : 23728 & 23727

LECTURER : Prof. Dr. Mustafa Ersel Kamaşak &
Prof. Dr. Gökhan İnce

GROUP MEMBERS:

150230007 : CELAL GÜNDOĞDU

150230008 : İSMAİL EFE ÇELİK

SPRING 2026

Contents

1	INTRODUCTION	1
2	MATERIALS AND METHODS	1
2.1	16-Bit Register	1
2.2	Register File	3
2.3	Address Register File	5
2.4	Data Register	7
2.5	Data Memory	8
2.6	Data Memory Unit	9
2.7	Instruction Register	10
2.8	Instruction Memory	11
2.9	Instruction Memory Unit	12
2.10	Arithmetic Logic Unit	13
2.11	ALU System	16
3	RESULTS	18
3.1	Register16bit	19
3.2	RegisterFile	19
3.3	AddressRegisterFile	20
3.4	InstructionRegister	20
3.5	InstructionMemoryUnit	21
3.6	DataRegister	21
3.7	DataMemoryUnit	22
3.8	ArithmeticLogicUnit	22
3.9	ArithmeticLogicUnitSystem	23
4	DISCUSSION	24
4.1	Design Approach and Modularity	24
4.2	Active-Low versus Active-High Conventions	24
4.3	ALU Flag Handling	24
4.4	Subtraction via Two's Complement	25
4.5	Multiplexer-Based Data Routing	25
4.6	Memory Organization	25
5	CONCLUSION	25
6	TASK DISTRIBUTION	27

1 INTRODUCTION

The goal of this project was to design and implement a simplified 16-bit processor datapath using the Verilog hardware description language. The datapath we built covers the fundamental building blocks that sit at the heart of most computer processors: registers that hold data, an arithmetic logic unit that performs operations on that data, memory units for storing instructions and data, and multiplexers that route information between all of these components.

We started from the smallest piece — a single 16-bit register — and worked our way up to the top-level module we call the ALU System, which ties every sub-module together into a functioning datapath. Along the way we implemented the Register File (containing four general-purpose registers and four scratch registers), the Address Register File (housing the Program Counter, Stack Pointer, and Address Register), a Data Register and Data Memory Unit for RAM access, an Instruction Register and Instruction Memory Unit for ROM access, and a 16-operation Arithmetic Logic Unit complete with flag logic. Each module was verified independently through test benches provided as part of the project, and finally the integrated ALU System passed all system-level tests as well.

The whole design was simulated in Xilinx Vivado 2017.4, and every single test case across all modules resulted in a pass, giving us confidence that the datapath behaves correctly under the conditions specified. In the sections that follow, we describe each module in detail, present the simulation results, and discuss the design decisions we made along the way.

2 MATERIALS AND METHODS

In this section we walk through every module that makes up the processor datapath. For each one we explain the purpose, the port list, and the internal logic we implemented. Where it helps, we include truth tables and code excerpts.

2.1 16-Bit Register

The 16-bit register is the most fundamental storage element in our design. Every other register-based module in the project — the Register File, the Address Register File, the Instruction Register — ultimately relies on instances of this component.

2.1.1 Port Description

Table 1: Port list of the 16-Bit Register module.

Port	Direction	Width	Description
Clock	Input	1	System clock signal
I	Input	16	Data input
E	Input	1	Enable (active high)
FunSel	Input	2	Function select
Q	Output	16	Register output

2.1.2 Function Selection

The register supports four operations selected by the 2-bit **FunSel** signal. All operations take effect on the rising edge of the clock and only when the enable signal **E** is high. If **E** is low, the register simply holds its current value regardless of **FunSel**.

Table 2: Function select truth table for the 16-Bit Register.

FunSel[1]	FunSel[0]	Operation
0	0	Clear — output is set to 16'h0000
0	1	Load — output becomes the value on input I
1	0	Increment — current value is increased by one
1	1	Decrement — current value is decreased by one

The implementation is straightforward: a **case** statement inside an **always @(posedge Clock)** block checks **FunSel** and updates **Q** through non-blocking assignments [1].

2.1.3 Schematic Diagram

Figure 1 shows the RTL schematic generated by Vivado for the 16-bit register. The structure reveals the main building blocks at the gate level: an adder (RTL_ADD) and a subtractor (RTL_SUB) compute the incremented and decremented values of the current output, while a 4-to-1 multiplexer (RTL_MUX) selects among the cleared value (ground), the loaded input, the incremented result, and the decremented result based on **FunSel[1:0]**. The selected value is driven into a 16-bit D flip-flop (RTL_REG) whose clock enable is connected to **E**. This confirms that the register holds its value when **E** is deasserted, regardless of **FunSel**.

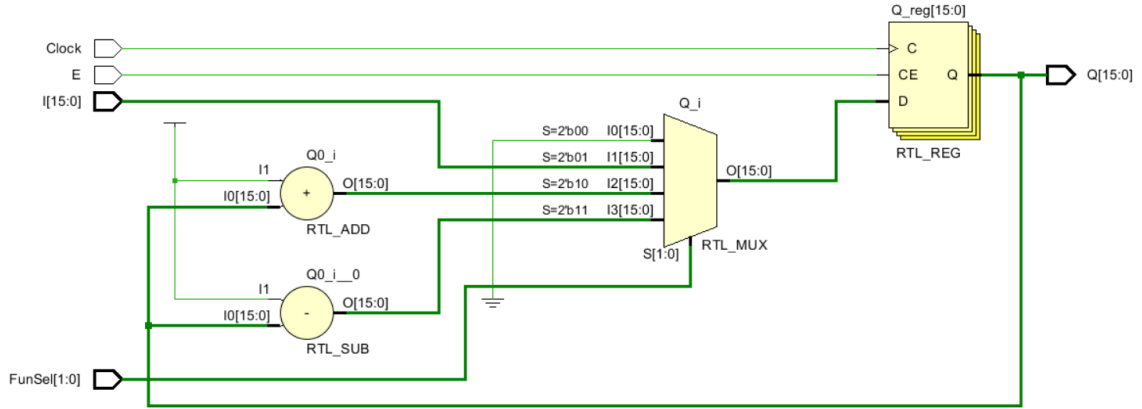


Figure 1: RTL schematic of the 16-Bit Register module.

2.2 Register File

The Register File groups eight 16-bit registers into one module and provides two independent read ports. Four of these registers are labeled as general-purpose (R1 through R4) and the remaining four serve as scratch registers (S1 through S4).

2.2.1 Port Description

Table 3: Port list of the Register File module.

Port	Direction	Width	Description
Clock	Input	1	System clock
I	Input	16	Shared data input for all registers
FunSel	Input	2	Function select (shared)
RegSel	Input	4	Enable for R1–R4 (active low)
ScrSel	Input	4	Enable for S1–S4 (active low)
OutASel	Input	3	Selects which register drives OutA
OutBSel	Input	3	Selects which register drives OutB
OutA	Output	16	First read port
OutB	Output	16	Second read port

2.2.2 Internal Structure

Internally, the module instantiates eight `Register16bit` blocks. The enable signal for each register is derived by inverting the corresponding bit of `RegSel` or `ScrSel` — for example, R1 is enabled when `RegSel[3]` is 0. This active-low convention means that a `RegSel` value of `4'b1111` disables all general-purpose registers, while `4'b0111` enables only R1.

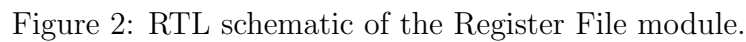
Two 8-to-1 multiplexers (implemented as a helper module called `Multiplexer3_8`) route the outputs of the eight registers to `OutA` and `OutB` according to `OutASel` and `OutBSel`. The selection mapping is given in Table 4. Because the two multiplexers are independent, the Register File can supply two different register values to the ALU in the same clock cycle [2].

Table 4: Output select mapping for the Register File.

<code>OutASel</code> / <code>OutBSel</code>	Selected Register
000	R1
001	R2
010	R3
011	R4
100	S1
101	S2
110	S3
111	S4

2.2.3 Schematic Diagram

Figure 2 presents the synthesized RTL view of the Register File. On the left side, eight `Register16bit` instances (R1–R4, S1–S4) are visible, each preceded by an inverter (`RTLINV`) that converts the active-low enable bits from `RegSel` and `ScrSel` into active-high enables for the underlying registers. On the right side, two cascaded multiplexer trees (labeled MA and MB) implement the 8-to-1 selection for `OutA` and `OutB`.



Whereas the Register File holds data for computations, the Address Register File stores addresses that the processor needs during execution. It contains three registers:

- ### 2.3.1 Port Description

Port	Direction	Width	Description
Clock	Input	1	System clock
I	Input	16	Shared data input
FunSel	Input	2	Function select (shared)
RegSel	Input	3	Enable bits — PC, SP, AR (active low)
OutCSel	Input	2	Selects the register for OutC
OutDSel	Input	1	Selects the register for OutD
OutC	Output	16	First read port
OutD	Output	16	Second read port
OutE	Output	16	Dedicated PC output

2.3.2 Output Multiplexing

OutC can carry the value of PC, AR, or SP depending on **OutCSel**. The mapping is shown in Table 6. **OutD** selects between AR and SP through the single-bit **OutDSel** ($0 \rightarrow \text{AR}$, $1 \rightarrow \text{SP}$). **OutE** is hard-wired to the Program Counter; it feeds directly into the Instruction Memory Unit so that instruction fetches always use the current PC value.

Table 6: OutCSel mapping for the Address Register File.

OutCSel[1]	OutCSel[0]	OutC Source
0	0	PC
0	1	PC
1	0	AR
1	1	SP

2.3.3 Schematic Diagram

Figure 3 shows the RTL schematic of the Address Register File. Three **Register16bit** instances — PC, AR, and SP — share the same **FunSel** and **I** buses. Each register’s enable comes from the corresponding inverted bit of **RegSel[2:0]**. Below the registers, three multiplexers handle the output routing: a two-level mux tree for **OutC** (first selecting between AR and SP based on **OutCSel[0]**, then between that result and PC based on **OutCSel[1]**), a single mux for **OutD**, and a direct wire from PC to **OutE**.

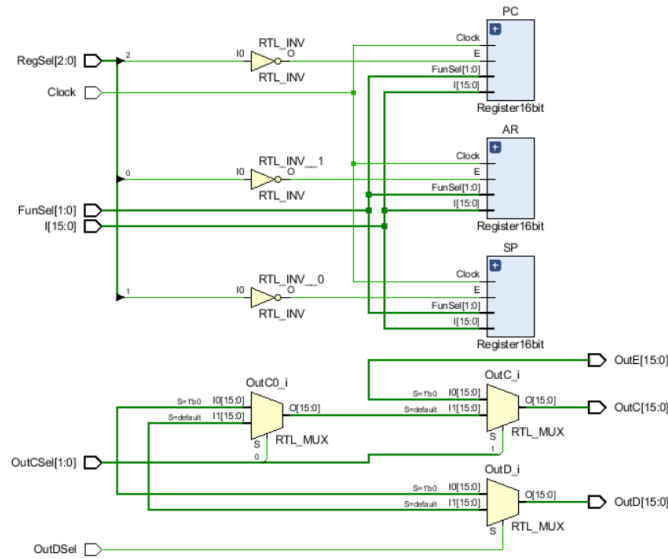


Figure 3: RTL schematic of the Address Register File module.

2.4 Data Register

Data memory in our system is byte-addressable, meaning each address points to an 8-bit value. However, the internal data buses of the processor are 16 bits wide. The Data Register bridges this gap by accepting 8-bit values from memory and assembling them into a 16-bit word.

2.4.1 Port Description

Table 7: Port list of the Data Register module.

Port	Direction	Width	Description
Clock	Input	1	System clock
I	Input	8	Byte input from memory
E	Input	1	Enable (active high)
FunSel	Input	1	Selects low or high byte
DROut	Output	16	16-bit composite output

2.4.2 Byte-Loading Mechanism

When **FunSel** is 0, the incoming byte is placed into the lower half of **DROut** (bits 7 down to 0), leaving the upper half unchanged. When **FunSel** is 1, the byte goes into the upper half (bits 15 down to 8). This way, two consecutive clock cycles — one with **FunSel** = 0 and one with **FunSel** = 1 — can load a full 16-bit word from byte-addressable memory. The behavior is summarized in Table 8.

Table 8: Functionality table of the Data Register.

E	FunSel	DROut ⁺ (Next State)
0	ϕ	DROut (Retain value)
1	0	DROut[15:8] unchanged, DROut[7:0] $\leftarrow$ I[7:0]
1	1	DROut[15:8] $\leftarrow$ I[7:0], DROut[7:0] unchanged

2.4.3 Schematic Diagram

The RTL schematic of the Data Register is shown in Figure 4. The key element is a pair of multiplexers controlled by **FunSel**: depending on its value, the 8-bit input **I** is routed to either the lower or upper byte lane of the 16-bit register (**RTL_REG**). The non-

selected byte retains its previous value by feeding back from the register output through the alternate mux path.

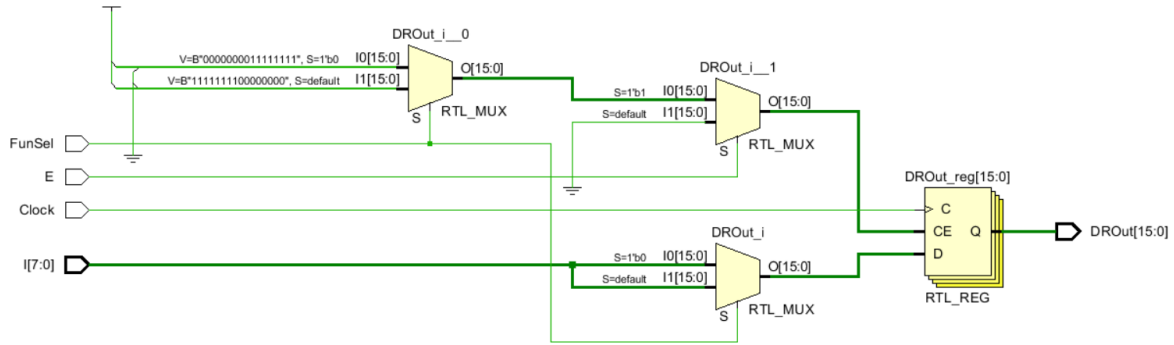


Figure 4: RTL schematic of the Data Register module.

2.5 Data Memory

The Data Memory module models a $64K \times 8$ -bit RAM. At start-up the memory contents are loaded from an external file (RAM.mem) using the `$readmemh` system task.

2.5.1 Port Description

Table 9: Port list of the Data Memory module.

Port	Direction	Width	Description
Clock	Input	1	System clock
Address	Input	16	Memory address
Data	Input	8	Data to be written
WR	Input	1	Write/Read control (1 = write, 0 = read)
CS	Input	1	Chip select (active low)
MemOut	Output	8	Data read from memory

2.5.2 Read and Write Behavior

Reads are handled combinatorially: whenever both `WR` and `CS` are low, `MemOut` reflects the byte stored at the given address. When either condition is not met, `MemOut` is driven to high impedance (`8'hZ`). Writes are synchronous: on the rising clock edge, if `WR` is

high and **CS** is low, the value on **Data** is stored into the addressed location. Table 10 summarizes the operating modes.

Table 10: Data Memory operating modes.

CS	WR	Operation
1	X	Disabled — MemOut = 8'hZ
0	0	Read — MemOut = RAM[Address]
0	1	Write — RAM[Address] $\leftarrow$ Data (on rising edge)

2.6 Data Memory Unit

The Data Memory Unit combines the Data Memory and the Data Register into a single wrapper module. It gives the rest of the system a clean interface: supply an address and chip-select, and the unit takes care of reading from RAM and assembling bytes into a 16-bit word through the Data Register.

2.6.1 Port Description

Table 11: Port list of the Data Memory Unit module.

Port	Direction	Width	Description
Clock	Input	1	System clock
I	Input	8	Byte input (write data)
Address	Input	16	Memory address
WR	Input	1	Write/Read control
CS	Input	1	Chip select (active high at this level)
FunSel	Input	1	Byte select for Data Register
DMUOut	Output	16	16-bit combined output

2.6.2 Internal Wiring

The module instantiates one **DataMemory** and one **DataRegister**. Note that the chip-select polarity is inverted inside this module — the **CS** input coming from the ALU System is active high, but the Data Memory expects an active-low chip select, so the module passes $\sim$ **CS** to the Data Memory's **CS** port. The **MemOut** wire from the Data Memory feeds directly into the Data Register's 8-bit input **I**. The enable of the Data Register is tied to **CS**, ensuring the register only latches new data when the unit is actually selected.

2.6.3 Schematic Diagram

Figure 5 illustrates the RTL view of the Data Memory Unit. The inverter (RTL_INV) on the CS line is clearly visible, converting the active-high chip select from the system level into the active-low signal expected by the DataMemory sub-module. The 8-bit MemOut bus connects the DataMemory output to the DataRegister input, and the 16-bit DROut is routed out as DMUOut.

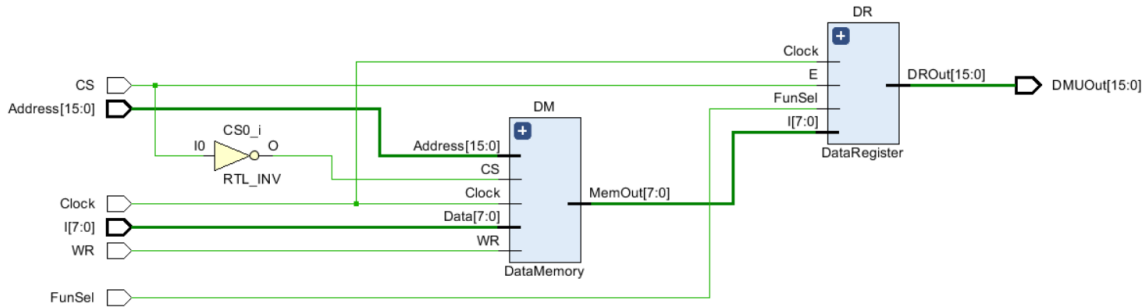


Figure 5: RTL schematic of the Data Memory Unit module.

2.7 Instruction Register

The Instruction Register is structurally similar to the Data Register: it accepts 8-bit values and stores them into either the lower or upper byte of a 16-bit output. The difference is in purpose — this register holds the current instruction fetched from ROM.

2.7.1 Port Description

Table 12: Port list of the Instruction Register module.

Port	Direction	Width	Description
Clock	Input	1	System clock
I	Input	8	Byte from instruction memory
Write	Input	1	Write enable
LH	Input	1	Low/High selector (0 = low byte, 1 = high byte)
IROut	Output	16	Full 16-bit instruction word

On the rising clock edge, if **Write** is asserted, the register loads **I** into either **IROut**[7:0] or **IROut**[15:8] depending on the value of **LH**. This allows a 16-bit instruction to be fetched from an 8-bit-wide ROM in two cycles. Table 13 summarizes the byte-loading behavior.

Table 13: Functionality table of the Instruction Register.

Write	LH	IROut ⁺ (Next State)
0	ϕ	IROut (Retain value)
1	0	IROut[15:8] unchanged, IROut[7:0] $\leftarrow$ I[7:0]
1	1	IROut[15:8] $\leftarrow$ I[7:0], IROut[7:0] unchanged

2.7.2 Schematic Diagram

The schematic for the Instruction Register is shown in Figure 6. Structurally it is nearly identical to the Data Register: a pair of multiplexers steered by the **LH** signal determines which byte position receives the incoming data, while the other byte position retains its value through feedback from the register output.

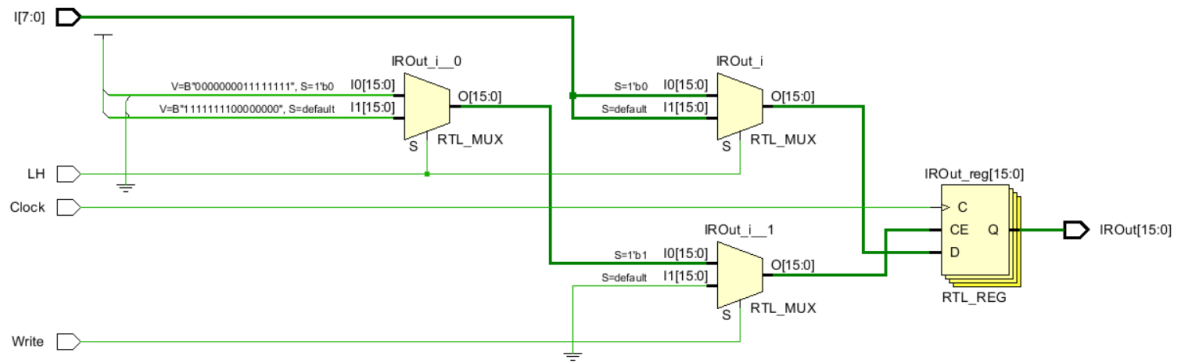


Figure 6: RTL schematic of the Instruction Register module.

2.8 Instruction Memory

The Instruction Memory is a read-only memory (ROM) that stores the program the processor is supposed to execute. Like the Data Memory, it is organized as $64K \times 8$ bits, and its contents are initialized from an external hex file (**ROM.mem**) at simulation start.

2.8.1 Port Description

Table 14: Port list of the Instruction Memory module.

Port	Direction	Width	Description
Address	Input	16	Memory address
MemOut	Output	8	Byte read from ROM

Because ROM is inherently read-only, there is no clock, chip-select, or write-enable port. The output is purely combinational: `MemOut` always reflects the content at the supplied address [3].

2.9 Instruction Memory Unit

The Instruction Memory Unit wraps the Instruction Memory and the Instruction Register together in the same way that the Data Memory Unit wraps its two components.

2.9.1 Port Description

Table 15: Port list of the Instruction Memory Unit module.

Port	Direction	Width	Description
Clock	Input	1	System clock
Address	Input	16	Address driven by the PC (OutE)
CS	Input	1	Chip select / write enable for the IR
LH	Input	1	Low/High byte select for the IR
IROut	Output	16	Full instruction word from the IR
IMUOut	Output	16	Zero-extended lower byte of the instruction

2.9.2 IMUOut Signal

An important detail of this module is the `IMUOut` signal. It takes the lower 8 bits of `IROut` and zero-extends them to 16 bits: `IMUOut = {8'd0, IROut[7:0]}`. This output can be used for immediate value extraction, where the lower byte of an instruction encodes a small constant that needs to be placed on a 16-bit bus.

2.9.3 Schematic Diagram

Figure 7 shows the RTL schematic of the Instruction Memory Unit. The `InstructionMemory` block (IM) is on the left; its `MemOut[7:0]` output feeds the `I[7:0]` input of the `InstructionRegister` block (IR). The `CS` and `LH` inputs are routed to the IR's `Write` and `LH` ports respectively. On the right-hand side, the `IROut[15:0]` bus is split: the full 16 bits go out as `IROut`, while only the lower 8 bits are zero-extended through an output buffer to produce `IMUOut[15:0]`.

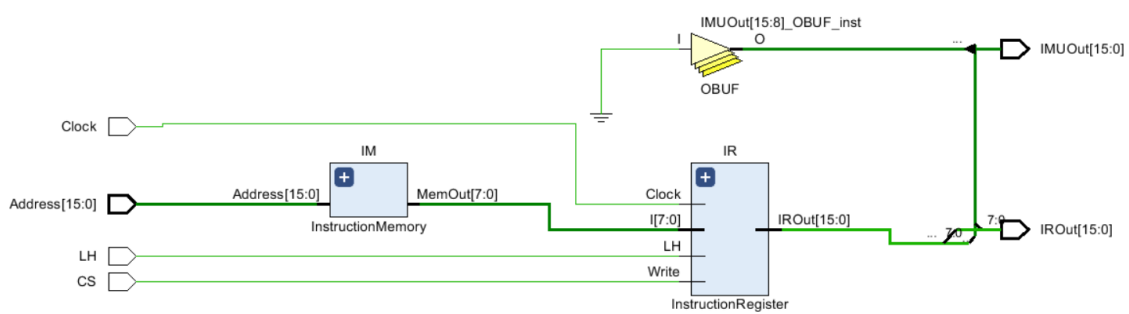


Figure 7: RTL schematic of the Instruction Memory Unit module.

2.10 Arithmetic Logic Unit

The ALU is the computational core of our datapath. It can perform 16 different operations on two 16-bit operands and produces a 16-bit result together with four condition flags.

2.10.1 Port Description

Table 16: Port list of the Arithmetic Logic Unit module.

Port	Direction	Width	Description
Clock	Input	1	System clock
A	Input	16	First operand (from OutA of RF)
B	Input	16	Second operand (from OutB of RF)
FunSel	Input	4	Operation select
WF	Input	1	Write flags enable
ALUOut	Output	16	Result of the selected operation
FlagsOut	Output	4	Condition flags: Z, C, N, O

2.10.2 Operation Table

Table 17: ALU operation table showing all 16 supported functions.

FunSel	Decimal	Operation
0000	0	Pass A
0001	1	Pass B
0010	2	Bitwise NOT A
0011	3	Bitwise NOT B
0100	4	$A + B$ (addition)
0101	5	$A + B + \text{Carry}$ (add with carry)
0110	6	$A - B$ (subtraction via two's complement)
0111	7	$A \text{ AND } B$
1000	8	$A \text{ OR } B$
1001	9	$A \text{ XOR } B$
1010	10	$A \text{ NAND } B$
1011	11	Logical shift left A (LSL)
1100	12	Logical shift right A (LSR)
1101	13	Arithmetic shift right A (ASR)
1110	14	Circular shift left A (CSL)
1111	15	Circular shift right A (CSR)

2.10.3 Flag Logic

The ALU maintains four flags packed into a 4-bit register `FlagsOut[3:0]`:

- **Z (Zero)** — **bit 3**: Set to 1 when the result is exactly zero.
- **C (Carry)** — **bit 2**: Updated by the addition, subtraction, and shift operations. For addition, it is the 17th bit of the extended sum. For subtraction (which is implemented as $A + \overline{B} + 1$), it similarly comes from the extended result. For shifts, the carry captures the bit that is shifted out of the register.
- **N (Negative)** — **bit 1**: Normally reflects the most significant bit of the result (bit 15). A special case exists for arithmetic right shift (**FunSel** = 13), where the N flag is left unchanged to preserve sign information.
- **O (Overflow)** — **bit 0**: Set when a signed overflow occurs during addition or subtraction. The overflow condition is detected by checking whether two operands of the same sign produce a result of the opposite sign.

The **ALUOut** is computed combinatorially, but the flags are latched into **FlagsOut** synchronously on the rising clock edge, and only when the **WF** signal is asserted. This separation ensures that the flag register is not inadvertently overwritten by operations that are merely passing data through the ALU without intending to affect the processor's condition codes [4].

Table 18 shows exactly which flags are affected by each category of ALU operation.

Table 18: Flag update behavior per ALU operation category.

Operation Category	Z	C	N	O
Pass / NOT / Logic (0–3, 7–10)	Updated	Unchanged	Updated	Unchanged
Add / Add+Carry (4, 5)	Updated	Updated	Updated	Updated
Subtract (6)	Updated	Updated	Updated	Updated
LSL / LSR / CSL / CSR (11,12,14,15)	Updated	Updated	Updated	Unchanged
ASR (13)	Updated	Unchanged	Unchanged	Unchanged

2.10.4 Schematic Diagram

Figure 8 presents the RTL schematic of the ALU. Due to the 16 different operations, the schematic is quite dense. The main data path flows from left to right: the two operand buses (**A[15:0]** and **B[15:0]**) enter from the bottom-left, and a large multiplexer tree (**RTL_MUX**) on the right selects the correct result based on **FunSel[3:0]**. The adder blocks (**RTL_ADD**) handle addition, subtraction (via the inverter **RTL_INV** on **B**), and the carry chain. At the bottom, several **RTL_ROM** blocks and an **RTL_EQ** comparator

compute the flag values, which are then gated by WF before being stored in the 4-bit flag register (RTL_REG, FlagsOut_reg).

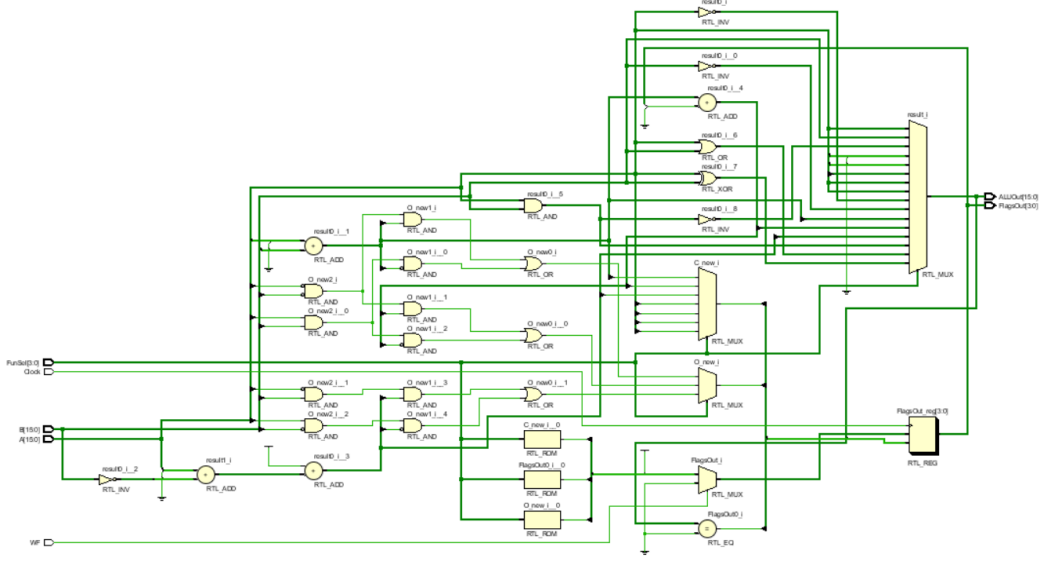


Figure 8: RTL schematic of the Arithmetic Logic Unit module.

2.11 ALU System

The ALU System is the top-level module that connects all previous components into a single, coherent datapath. It is essentially the backbone of our simplified processor.

2.11.1 Architecture Overview

The module instantiates one Register File, one Address Register File, one ALU, one Instruction Memory Unit, and one Data Memory Unit. Three multiplexers (**MuxA**, **MuxB**, and **MuxC**) route data between these blocks:

- **MuxA** (2-bit select, **MuxASel**): Feeds the Register File's data input. It can choose among the ALU output, the Address Register File's **OutC**, the Data Memory Unit output (**DMUOut**), or the Instruction Register output (**IROut**).
- **MuxB** (2-bit select, **MuxBSel**): Feeds the Address Register File's data input. It has the same four source options as MuxA.
- **MuxC** (1-bit select, **MuxCSel**): Selects either the lower byte (**ALUOut[7:0]**) or the upper byte (**ALUOut[15:8]**) to feed the Data Memory Unit's write data port.

2.11.2 Data Flow

The overall data flow can be summarized as follows. The Register File's two output ports (**OutA**, **OutB**) supply operands to the ALU. The ALU processes these operands and

produces a result on `ALUOut`. This result can be routed back to the Register File through `MuxA`, to the Address Register File through `MuxB`, or byte-by-byte to the Data Memory Unit through `MuxC`.

The Address Register File's `OutE` (always carrying the PC value) drives the address input of the Instruction Memory Unit, enabling instruction fetch. Its `OutD` drives the address input of the Data Memory Unit, facilitating data load and store operations. The `OutC` output can feed back into `MuxA` or `MuxB`, allowing addresses to be transferred to the Register File or to other address registers.

2.11.3 Control Signals

The ALU System does not implement a control unit — that falls outside the scope of this project. Instead, each control signal is exposed as a top-level input, and the test bench directly drives these signals to exercise specific data-paths within the system. Table 19 summarizes all control inputs.

Table 19: Control signals of the ALU System module.

Signal Group	Signal	Width
Register File	RF_OutASel	3
	RF_OutBSel	3
	RF_FunSel	2
	RF_RegSel	4
	RF_ScrSel	4
ALU	ALU_FunSel	4
	ALU_WF	1
Address Register File	ARF_OutCSel	2
	ARF_OutDSel	1
	ARF_FunSel	2
	ARF_RegSel	3
Instruction Memory Unit	IMU_CS	1
	IMU_LH	1
Data Memory Unit	DMU_WR	1
	DMU_CS	1
	DMU_FunSel	1
Multiplexers	MuxASel	2
	MuxBSel	2
	MuxCSel	1

3 RESULTS

We verified every module through the simulation test benches provided with the project files. All simulations were run on Xilinx Vivado 2017.4 using the batch script `Run.bat`. The summary of results is given in Table 20.

Table 20: Summary of simulation results for all modules.

Module	Tests Run	Passed	Failed
Register16bit	4	4	0
RegisterFile	4	4	0
AddressRegisterFile	5	5	0
InstructionRegister	2	2	0
InstructionMemoryUnit	5	5	0
DataRegister	2	2	0
DataMemoryUnit	3	3	0
ArithmeticLogicUnit	20	20	0
ArithmeticLogicUnitSystem	35	35	0
Total	80	80	0

All 80 individual checks across every module passed without a single failure. In the subsections that follow, we present the Vivado simulation waveform for each module and highlight a few noteworthy observations from the test runs.

3.1 Register16bit

The test bench verified that the register correctly holds its value when the enable signal is low (Test 1 and Test 3) and correctly clears or loads a new value when enable is high (Test 2 and Test 4). For example, in Test 2 the register was set to 0x0025, then a clear operation ($\text{FunSel} = 00$) was applied with $E = 1$. After the clock edge, the output correctly became 0x0000. In Test 4, the value 0x0012 was loaded via $\text{FunSel} = 01$, confirming that the load path is functional.

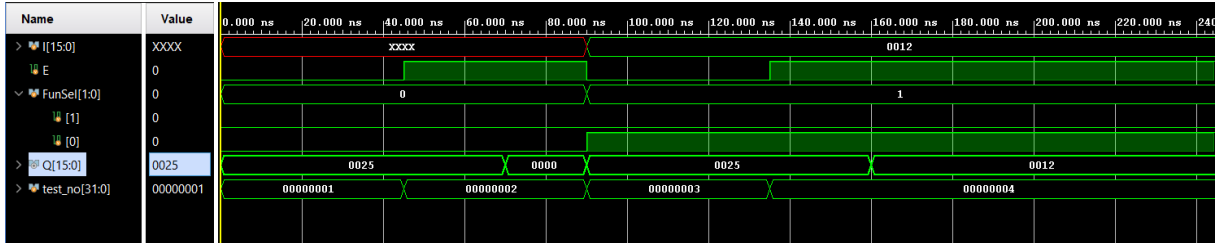


Figure 9: Simulation waveform for the Register16bit module.

3.2 RegisterFile

The Register File was tested by loading 0x1234 into R1 and 0x5678 into S2 in the first test, then reading both values through OutA and OutB. In Test 2, the output selectors were

changed to read R1 on OutA (value 0x1234) and R3 on OutB (value 0x3456), confirming the independence of the two read ports and the correctness of the output multiplexers.

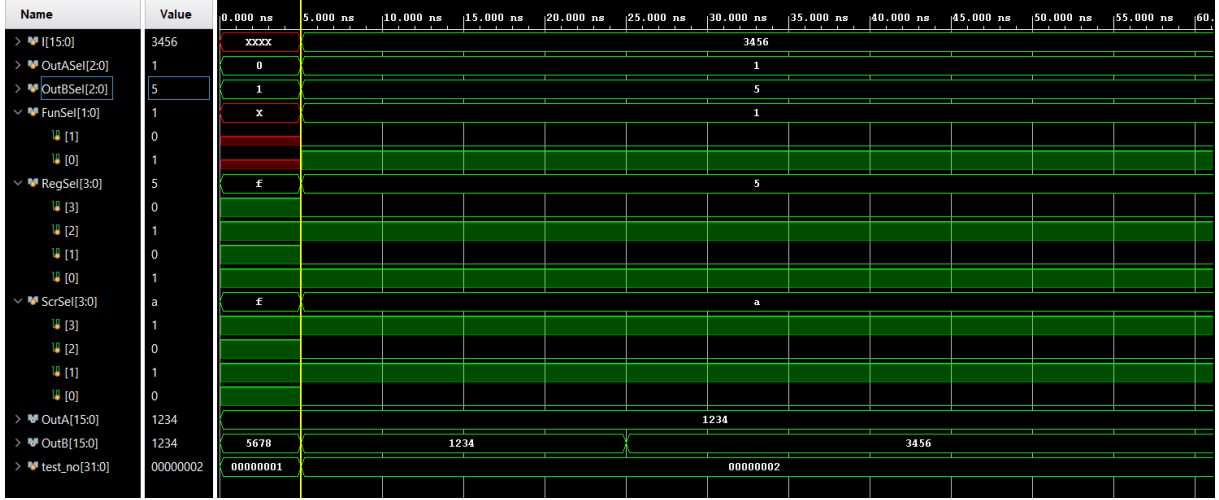


Figure 10: Simulation waveform for the RegisterFile module.

3.3 AddressRegisterFile

Five individual checks covered loading values into the PC and AR, verifying that OutC and OutD reflect the correct registers based on OutCSel and OutDSel, and confirming that OutE always tracks the PC value. In Test 2, after an increment operation on the loaded value 0x3548, OutE correctly updated to 0x3548, matching OutC and OutD.

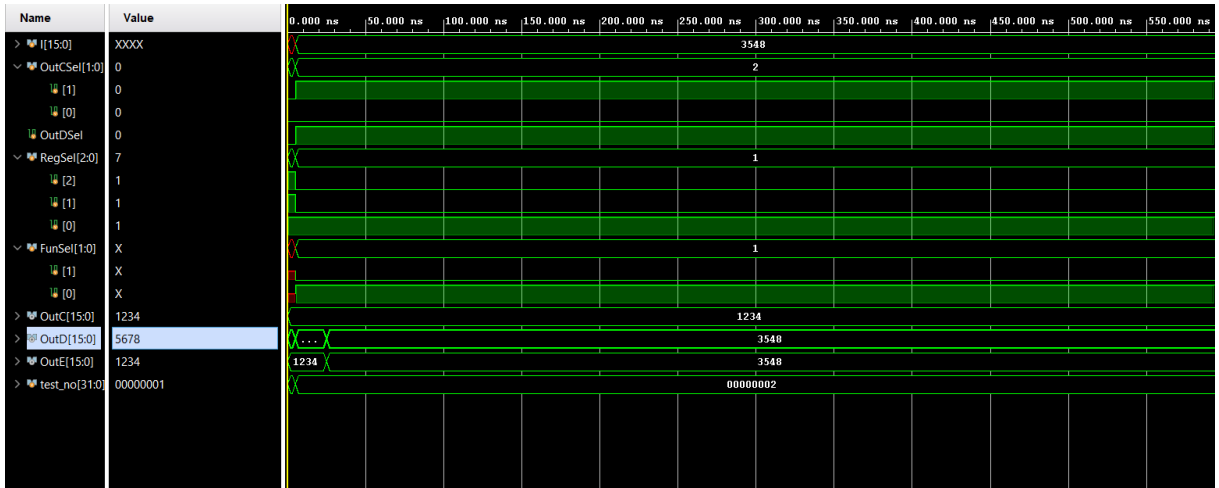


Figure 11: Simulation waveform for the AddressRegisterFile module.

3.4 InstructionRegister

The Instruction Register test loaded the lower byte first (LH = 0), then the upper byte (LH = 1). After two clock cycles, IROut correctly assembled the full 16-bit word 0x2315

in Test 1. Test 2 repeated the process with different data, yielding 0x1567.

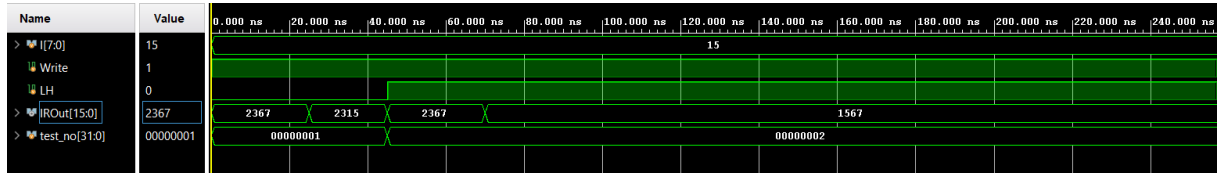


Figure 12: Simulation waveform for the InstructionRegister module.

3.5 InstructionMemoryUnit

The IMU test verified the full fetch pipeline. In Test 1, address 0x0015 was presented to the ROM, which returned 0x05. After the clock edge with CS = 1 and LH = 0, the lower byte of the IR was loaded, giving IROut = 0x2305 and IMUOut = 0x0005. In Test 2, the high byte of a different instruction was loaded, and IROut updated to 0x0567.

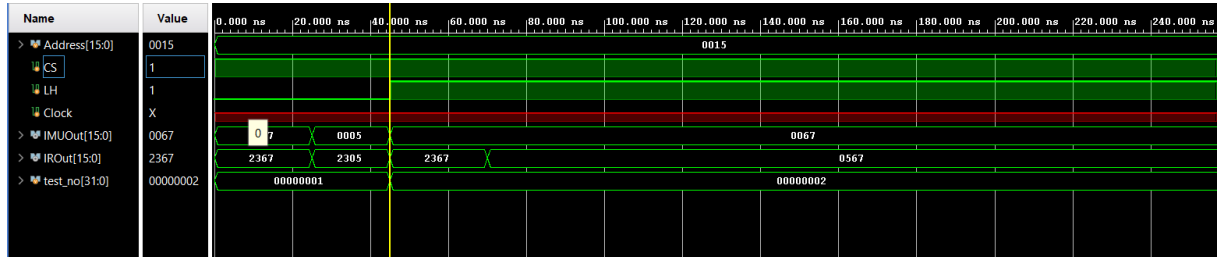


Figure 13: Simulation waveform for the InstructionMemoryUnit module.

3.6 DataRegister

The Data Register tests mirrored the Instruction Register tests. Byte 0x15 was loaded into the low byte (FunSel = 0), producing DROut = 0x2315. Then the same byte was loaded into the high position (FunSel = 1), resulting in DROut = 0x1567.

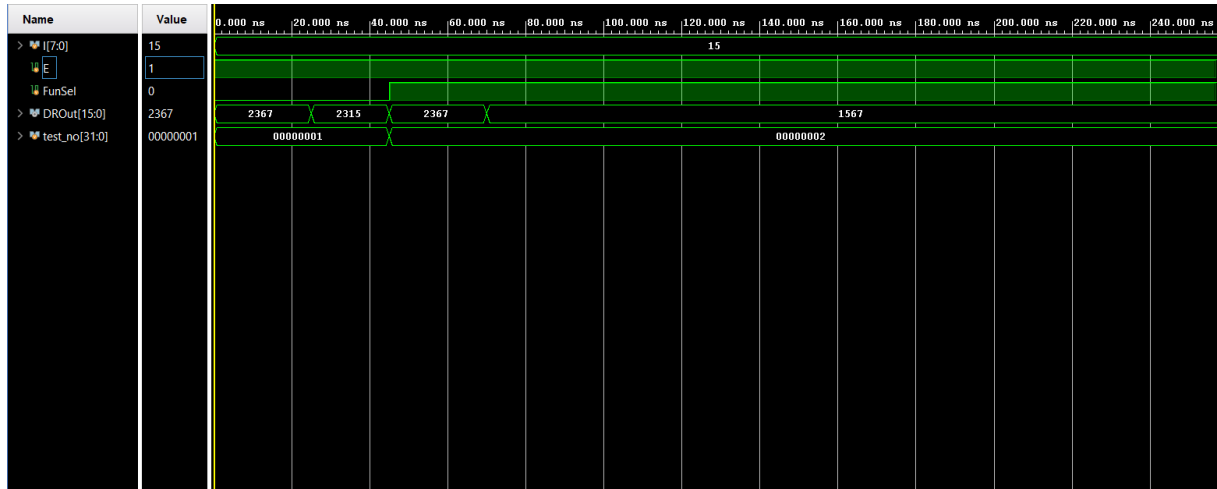


Figure 14: Simulation waveform for the DataRegister module.

3.7 DataMemoryUnit

The DMU test bench read from address 0x0015 and verified that the value 0x07 was fetched from RAM. After assembling two bytes through the Data Register, DMUOut correctly showed 0x2307 in Test 1 and 0x0767 in Test 2.

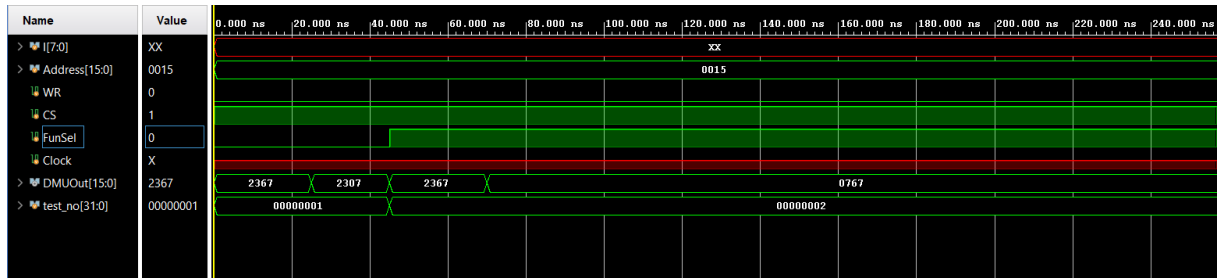


Figure 15: Simulation waveform for the DataMemoryUnit module.

3.8 ArithmeticLogicUnit

The ALU tests covered addition (FunSel = 4), add-with-carry (FunSel = 5), and pass-through (FunSel = 0). In Test 1, A = 0x1234 and B = 0x4321 were added, producing ALUOut = 0x5555 with all flags initially latched as 1 from the pre-set state. Test 3 verified the add-with-carry: 0x7777 + 0x8888 + 1 = 0x0000, correctly setting Z and C while clearing N and O. Test 4 confirmed that the pass-through operation on A = 0x2233 did not alter the ALU output, and the flags from the previous cycle were latched accurately on the next clock edge.

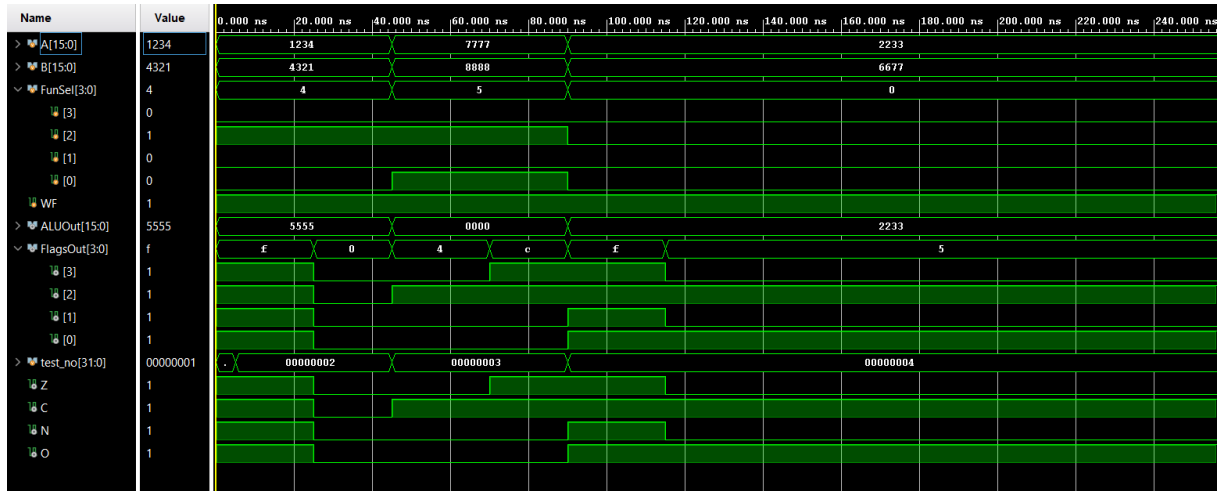


Figure 16: Simulation waveform for the ArithmeticLogicUnit module.

3.9 ArithmeticLogicUnitSystem

The system-level test bench verified multi-component interactions. In Tests 1 and 2, the Register File's R1 was loaded with 0x7777 and S2 with 0x8887. After an add-with-carry through the ALU, the result 0xFFFFE was routed via MuxA to load into R2 and S3, and simultaneously through MuxB to update the PC in the Address Register File. After the clock edge (Test 2), all three target registers (R2, S3, PC) correctly held 0xFFFFE.

Tests 3 and 4 exercised the instruction fetch path. The PC was set to 0x0023, and the ROM contained the value 0x15 at that address. With IMU_CS = 1 and IMU_LH = 0, the byte was fetched from ROM and, after a clock edge, loaded into the lower byte of the Instruction Register, giving IROut = 0x0015.

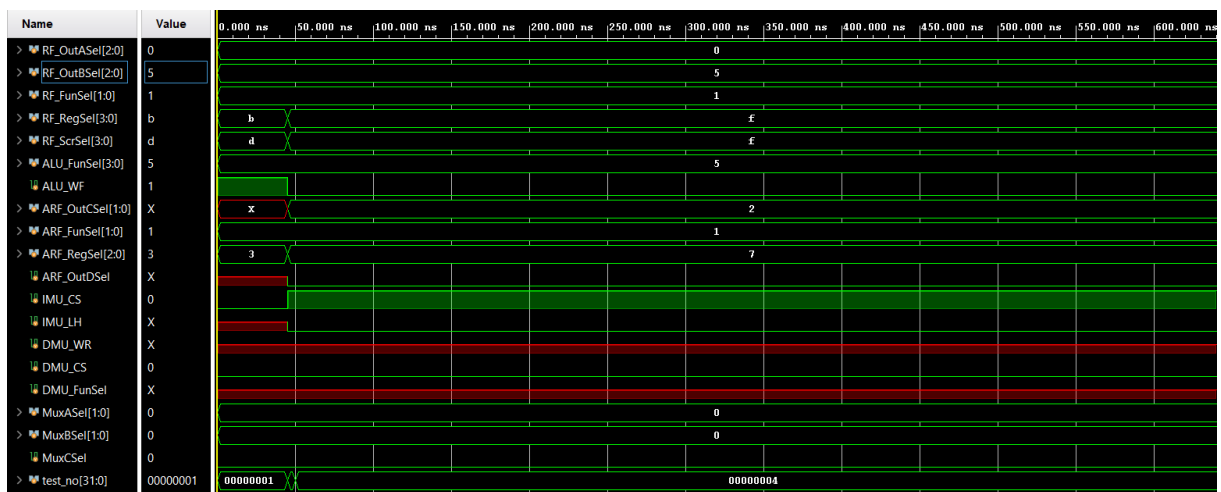


Figure 17: Simulation waveform for the ALU System module.

4 DISCUSSION

4.1 Design Approach and Modularity

We deliberately followed a bottom-up design methodology. We began with the simplest building block — the 16-bit register — and tested it thoroughly before moving on to the more complex aggregates. This approach made debugging far more manageable because, by the time we reached the ALU System, we already had high confidence in each sub-component. When something went wrong at the system level, we could focus on the interconnections rather than suspecting the individual modules.

4.2 Active-Low versus Active-High Conventions

One aspect that required careful attention was the mixing of active-low and active-high conventions across the design. The Register File and Address Register File use active-low enable signals (`RegSel`, `ScrSel`, `ARF_RegSel`), meaning a bit value of 0 enables the corresponding register. However, the Data Memory Unit's `CS` signal is active-high at the system level but gets inverted inside the module before being passed to the Data Memory (which expects active-low). Keeping track of these polarities was one of the trickier parts of the integration, and a small mistake here would silently let data flow into the wrong register or fail to write to memory.

4.3 ALU Flag Handling

The decision to update flags synchronously (on the clock edge) rather than combinatorially was a deliberate choice dictated by the project specification. This means that the combinational outputs `ALUOut` and the internally computed flag values are available immediately, but the `FlagsOut` register only updates on the next clock edge when `WF = 1`. This behavior is typical of real processor designs where a dedicated flag register preserves condition codes until the next instruction explicitly requests a flag update [5].

We paid special attention to the arithmetic right shift case (`FunSel = 13`). In this operation, the N flag must remain unchanged because the sign bit is preserved by definition. Not handling this edge case would cause the N flag to be overwritten with a value that is always the same as the existing MSB, which is correct in most cases but technically violates the specification in scenarios where the flag was previously set by a different operation.

4.4 Subtraction via Two's Complement

Rather than implementing a dedicated subtractor, we perform subtraction using the two's complement method: $A - B = A + \overline{B} + 1$. This is a well-known technique that allows a single adder circuit to handle both addition and subtraction, saving hardware resources [3]. In our Verilog implementation, we extend the operands to 17 bits before adding, which gives us the carry-out for free as the 17th bit of the sum.

4.5 Multiplexer-Based Data Routing

The three multiplexers in the ALU System serve as the system's "traffic controllers." Without a control unit (which would be the subject of a future project), these multiplexers allow the test bench to exercise any data path within the system by simply setting the appropriate select signals. The current architecture supports several common micro-operations:

- ALU result $\rightarrow$ Register File (MuxASel = 00)
- ALU result $\rightarrow$ Address Register File (MuxBSel = 00)
- Memory data $\rightarrow$ Register File (MuxASel = 10)
- Instruction register content $\rightarrow$ Register File (MuxASel = 11)
- Address register content $\rightarrow$ Register File (MuxASel = 01)

This flexibility means the datapath is general enough to support a wide variety of instruction types once a control unit is eventually added.

4.6 Memory Organization

Both the instruction and data memories are modeled as $64K \times 8$ -bit arrays, reflecting a byte-addressable memory architecture with a 16-bit address space. This gives a total addressable space of 65,536 bytes (64 KB). The use of `$readmemh` for initialization allows us to prepare memory contents in simple hexadecimal text files (`ROM.mem` and `RAM.mem`), which is convenient for setting up test scenarios without modifying the Verilog source.

5 CONCLUSION

In this project we successfully designed and implemented a 16-bit processor datapath consisting of nine Verilog modules, all of which passed their respective test benches with

zero failures out of 80 total checks. The work covered registers, register files, memory units, an ALU with 16 operations and 4 condition flags, and a top-level system module that ties everything together.

From a learning perspective, this project reinforced several key concepts. Working through the flag logic of the ALU — especially the overflow detection for signed addition and the special handling of the N flag during arithmetic right shift — helped us internalize how real processors maintain condition codes. The integration phase, where we had to keep straight active-low versus active-high polarities across module boundaries, was a practical lesson in the importance of clear interface specifications.

One difficulty we encountered involved the byte-loading mechanism for the Data Register and Instruction Register. It was initially not immediately obvious that two separate clock cycles were needed to assemble a full 16-bit word from byte-addressable memory, and getting the `FunSel` / `LH` signals correct for each half took a few iterations of checking the waveforms in simulation.

Overall, the project was a valuable hands-on exercise in digital system design. Having a working datapath makes the prospect of adding a control unit in a future project feel much more concrete — we now have a solid understanding of what signals the controller would need to drive, and in what sequence, to execute actual instructions.

6 TASK DISTRIBUTION

Table 21: Task distribution between group members.

Group Member	Tasks
İsmail Efe Çelik (150230008)	16-Bit Register module implementation and testbench verification
	Register File module implementation
	Address Register File module implementation
	Data Register and Data Memory module implementation
	Simulation and waveform analysis for the above modules
Celal Gündoğdu (150230007)	Instruction Register and Instruction Memory Unit implementation
	Data Memory Unit module implementation
	Arithmetic Logic Unit implementation (operations and flag logic)
	ALU System top-level integration and testbench verification
	Report writing, schematic figures, and documentation

REFERENCES

- [1] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.
- [2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2012.
- [3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [4] M. M. Mano and C. R. Kime, *Logic and Computer Design Fundamentals*, 5th ed. Hoboken, NJ, USA: Pearson, 2017.
- [5] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 10th ed. Hoboken, NJ, USA: Pearson, 2016.